2. Protocol Specification

Amazon–UPS Interoperability Protocol (AUIP) v1.0 (HTTP REST Version)

2.1 Purpose

This document specifies the RESTful HTTP protocol used for communication between the Amazon server and the UPS server in the interoperability group.

The protocol coordinates shipment creation, pickup scheduling, truck loading, package delivery, delivery status queries, and address redirection.

This protocol is independent from the World Simulator protocol. Amazon and UPS MUST each separately communicate with the World Simulator using the provided protobuf messages, while Amazon and UPS communicate with each other using the protocol defined in this document.

2.2 Design Goals

The protocol is designed to satisfy the following goals:

1. The protocol MUST support the end-to-end workflow:
   customer purchase → shipment creation → warehouse packing → truck pickup → truck loading → delivery.
2. The protocol MUST support asynchronous processing.
3. The protocol MUST support idempotent operations.
4. The protocol MUST ensure unique shipment identifiers within a world.
5. The protocol SHOULD support UPS account binding.
6. The protocol SHOULD support destination redirection before final delivery.
7. The protocol MUST tolerate duplicate HTTP requests caused by retry or timeout.

2.3 Transport

2.3.1 Network

- Communication MUST use HTTP/1.1 or HTTP/2 over TCP.
- UPS MUST expose REST endpoints.
- Amazon MUST act as the client and invoke UPS APIs.
- UPS hostname and port MUST be configurable.

2.3.2 Data Format

- All requests and responses MUST use `application/json`.
- Encoding MUST be UTF-8.

2.3.3 Timeouts

- Senders MAY retry on timeout or network error. Receivers MUST process such retries safely using idempotency rules defined below.

2.4 Idempotency

- All POST/PUT requests MUST include an `Idempotency-Key` header.
- UPS MUST ensure duplicate requests with the same key do not create duplicate effects.
- All state-changing POST and PUT requests MUST include an Idempotency-Key header.
- The receiver MUST treat the pair (HTTP method, path, Idempotency-Key) as the request identity.
- If the same request identity is received again with the same semantic request body, the receiver MUST return the same logical result and MUST NOT apply the state transition twice.
- If the same request identity is received again with a different request body, the receiver MUST reject it with 409 Conflict.
- Both Amazon and UPS MUST implement these rules. This requirement applies to callbacks in addition to public API endpoints.
- Receivers SHOULD persist idempotency records for at least the lifetime of the shipment.

2.5 Resource Model

Core Resource

```
1. Shipment
```

Each shipment is uniquely identified by:

```
shipment_id
```

```
2. World(identified by a 64-bit number)
```

Each world is uniquely identified by:

```
world_id
```

2.6 API Endpoints

2.6.0 Error Format

```
{
  "error_code": "INVALID_STATE",
  "message": "Shipment cannot be loaded before it is ready and truck-arrived.",
  "shipment_id": "PKG-10001"
}
```

2.6.1 Create Shipment(Amazon → UPS)

Endpoint

```
POST /shipments
```

Request Headers

```
Content-Type: application/json
Idempotency-Key: <uuid>
```

Request Body

```
{
  "shipment_id": "PKG-10001",
  "world_id": 101,
  "amazon_order_id": "ORD-1",
  "warehouse": {
    "id": 1,
    "x": 2,
    "y": 4
  },
  "destination": {
    "x": 9,
    "y": 8
  },
  "items": [
    {
      "product_id": 11,
      "description": "laptop",
      "quantity": 1
```

```
      }
    ]
}
```

Response

Success Response

201 Created

```
{
  "shipment_id": "PKG-10001",
  "status": "created",
  "tracking_number": "PKG-10001",
  "world_id": 123456789
}
```

Error Responses

400 Bad Request
409 Conflict

Rules

shipment_id MUST be unique within a world.
UPS MUST persist the shipment before returning success.
Duplicate requests MUST be handled idempotently.
UPS MUST validate all required fields before accepting the shipment.
UPS MUST initialize the shipment state as created.

2.6.2 Request Pickup(Amazon → UPS)

Endpoint

```
POST /shipments/{shipment_id}/pickup
```

Request Body

```
{
  "warehouse_id": 1,
  "priority": "normal",
  "world_id": 123456789
}
```

Response

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "truck_en_route",
  "world_id": 123456789
}
```

Rules

Amazon MUST create the shipment before requesting pickup.
UPS MUST dispatch a truck via the World Simulator.
Duplicate pickup requests MUST NOT create multiple truck dispatches.
UPS MAY accept pickup even if the package is not yet packed.
UPS MUST reject the request with 409 Conflict if the shipment is already loaded, out_for_delivery, delivered, or canceled.

2.6.3 Notify Truck Arrived (UPS → Amazon Callback)

Endpoint (Amazon side)

```
POST /api/truck-arrived
```

Request Body

```
{
  "shipment_id": "PKG-10001",
  "truck_id": 5,
  "warehouse_id": 1,
  "location": {
    "x": 2,
    "y": 4
  },
  "world_id": 123456789
}
```

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "truck_arrived",
  "world_id": 123456789
}
```

Rules

UPS MUST send this callback after the World Simulator confirms truck arrival at the warehouse.
Amazon MUST persist truck arrival before returning success.
Amazon MUST treat duplicate callbacks idempotently.
Amazon MUST reject with 409 Conflict if warehouse_id does not match the shipment.
Amazon MAY update status to truck_arrived even if the package is still packing.

2.6.4 Notify Package Ready (Amazon → UPS)

Endpoint

```
POST /shipments/{shipment_id}/ready
```

Request Body

```
{
  "warehouse_id": 1,
  "world_id": 123456789
}
```

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "packed",
  "world_id": 123456789
}
```

Rules

Amazon MUST send this request only after the package is fully packed and available for loading.
UPS MUST persist this state before returning success.
Duplicate requests MUST be idempotent.

UPS MUST reject with 409 Conflict if the shipment is already loaded, out_for_delivery, delivered, or canceled.

2.6.5 Request Load (UPS → Amazon Callback)

Endpoint (Amazon side)

```
POST /api/load-request
```

Request Body

```
{
  "shipment_id": "PKG-10001",
  "truck_id": 5,
  "warehouse_id": 1,
  "world_id": 123456789
}
```

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "loading",
  "world_id": 123456789
}
```

Rules

UPS MUST send this callback only after truck arrival and only when it is ready to receive the package.
Amazon MUST reject with 409 Conflict if the package is not packed or if the truck has not arrived.
Amazon MUST reject with 409 Conflict if warehouse_id or truck_id does not match expected shipment state.
Duplicate requests MUST be handled idempotently.

2.6.6 Confirm Loaded (Amazon → UPS)

Endpoint

```
POST /shipments/{shipment_id}/loaded
```

Request Body

```
{
  "truck_id": 5,
  "warehouse_id": 1,
  "world_id": 123456789
}
```

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "loaded",
  "world_id": 123456789
}
```

Rules

Amazon MUST send this request only after successful loading in the World Simulator.
Amazon MUST NOT call this endpoint before the package is packed and the truck has arrived.
UPS MUST trigger delivery scheduling only after receiving this confirmation.
Duplicate requests MUST NOT trigger multiple deliveries.

2.6.7 Notify Out for Delivery (UPS → Amazon)

Endpoint (Amazon side)

```
POST /api/out-for-delivery
```

Request Body

```
{
  "shipment_id": "PKG-10001",
  "truck_id": 5,
  "destination": {
    "x": 9,
    "y": 8
  },
  "world_id": 123456789
}
```

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "out_for_delivery",
  "world_id": 123456789
}
```

Rules

UPS MUST send this callback after starting delivery in the World Simulator.
UPS MUST NOT send this callback before the shipment is loaded.
Amazon MUST persist the new status before returning success.
Duplicate callbacks MUST be idempotent.

2.6.8 Notify Delivered (UPS → Amazon)

Endpoint (Amazon side)

```
POST /api/delivered
```

Request Body

```
{
  "shipment_id": "PKG-10001",
  "truck_id": 5,
  "location": {
    "x": 9,
    "y": 8
  },
  "world_id": 123456789
}
```

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "delivered",
  "world_id": 123456789
}
```

Rules

UPS MUST send this callback after the World Simulator confirms the package is delivered.
Amazon MUST mark the shipment as delivered.
Both sides MUST treat duplicate delivered callbacks as idempotent.
A shipment in delivered MUST NOT transition back to any earlier state.

2.6.9 Query Shipment Status(UPS & Amazon)

Endpoint

```
GET /shipments/{shipment_id}
```

Response

Success Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "out_for_delivery",
  "truck_id": 5,
  "warehouse_id": 1,
  "destination": {
    "x": 9,
    "y": 8
  },
  "ups_account": "bob123",
  "world_id": 123456789
}
```

Error Response

404 Not Found

Rules

UPS MUST return the current persisted state of the shipment.
This endpoint MUST be read-only.
This endpoint does not require an Idempotency-Key.

2.6.10 Update Destination(UPS & Amazon)

Endpoint

```
PUT /shipments/{shipment_id}/destination
```

Request Body

```
{
  "x": 50,
  "y": 80,
  "reason": "customer_redirect"
}
```

Response

200 OK

```
{
  "shipment_id": "PKG-10001",
  "status": "updated",
  "destination": {
    "x": 50,
    "y": 80
  },
  "world_id": 123456789
}
```

Rules

UPS MUST reject the request with 409 Conflict if shipment status is out_for_delivery, delivered, or canceled.
UPS MUST reject invalid coordinates with 400 Bad Request.
Duplicate update requests MUST be idempotent.
If account ownership is implemented, UPS MUST reject destination updates for non-owners with 403 Forbidden or 409 Conflict, depending on local implementation policy.

2.7 Shipment Status Values

The following status values MUST be used:

- `created`
- `packing`
- `packed`
- `truck_en_route`
- `truck_arrived`
- `loading`
- `loaded`
- `out_for_delivery`
- `delivered`
- `delivery_failed`
- `canceled`
- `updated`

2.8 Required End-to-End Flow

1. Amazon creates a shipment with POST /shipments.
2. UPS persists the shipment and returns 201 Created.
3. Amazon requests pickup with POST /shipments/{shipment_id}/pickup.
4. UPS dispatches a truck through the World Simulator.
5. UPS receives truck-arrival confirmation from the World Simulator.
6. UPS notifies Amazon via POST /api/truck-arrived.
7. Amazon packs the shipment and sends POST /shipments/{shipment_id}/ready.
8. UPS requests package load using POST /api/load-request.
9. Amazon loads the shipment in the World Simulator.
10. Amazon confirms load with POST /shipments/{shipment_id}/loaded.
11. UPS dispatches delivery in the World Simulator.
12. UPS notifies Amazon via POST /api/out-for-delivery.
13. UPS receives delivery confirmation from the World Simulator.
14. UPS notifies Amazon via POST /api/delivered

2.9 State Constraints

Amazon

- MUST NOT call `/loaded` before package is ready
- MUST NOT load before truck arrival

UPS

- MUST NOT deliver before `/loaded`
- MUST ensure shipment uniqueness

2.10 Reliability

- All operations MUST be idempotent
- UPS MUST handle duplicate requests
- Amazon MAY retry on timeout

2.11 Persistence

Both systems MUST persist:

- shipment_id
- world_id
- status
- destination
- warehouse
- ups_account
- truck_id

2.12 Security

- No user-level authentication is required for the interoperability protocol in this class project.
- Each service SHOULD restrict access to known peer hostnames, IP addresses, or docker-network peers where practical.
- Each service MUST validate all inputs.
- Callback endpoints SHOULD NOT be exposed publicly beyond the deployment network unless explicitly intended.

2.13 Compliance Requirements

Amazon MUST:

- create shipment before requesting pickup
- notify UPS when package is ready
- notify UPS when package is loaded
- reject invalid load requests
- support shipment state persistence
- handle duplicate UPS callbacks idempotently

UPS MUST:

- create and persist shipment records
- dispatch trucks
- notify Amazon of truck arrival

- notify Amazon of out-for-delivery
- notify Amazon of delivery completion
- support shipment status query
- support duplicate request detection
- reject invalid state transitions
